Today: Derandomization 2

In fact, there exists a whole bunch of cryptographic primitives, whose existence is equivalent. AFAIK, this includes: one way functions, private key crypto, PRGs (in the crypto regime, i.e. with n^c stretch and computable in poly(n) time), and a bunch of others

Recap: baby step of NW PRG

Lemma. Suppose that $f : \{0,1\}^n \to \{0,1\}$ has $\mathsf{H}_{\mathsf{avg}}(f) \geq n^4$, say. Then the generator

$$G : \{0,1\}^{\log n + 2(n - \log n)} \to \{0,1\}^{\log n + 2(n-\log n)+2}$$
$$G(x_0, x_1, x_2) = x_0 x_1 x_2 f(x_0 x_1) f(x_0 x_2)$$

is $(n^4/2 - n/2 - 10, n^{-3})$-secure.

What do we see here? We see that we apply the same functions on two different parts of the input (x_0x_1 and x_0x_2) that share some of the bit positions, namely x_0.

To prove that this is secure, we can use Yao's unpredictability theorem for circuits. We have seen last time in class that $x_0x_1x_2f(x_0x_1)$ are unpredictable, under the assumption that f is hard on average. Let us now repeat the proof that the last bit is also unpredictable.

Again, as usual, we go by the contrapositive: we assume we have a predictor for the last bit, and we construct a circuit that computes f on average.

So, let's suppose that there exists a predictor P, namely a circuit of size S which can predict the last bit when given the previous bits:

$$\Pr_{x_0x_1x_2}[P(x_0, x_1, x_2, f(x_0x_1)) = f(x_0x_2)] > \frac{1}{2} + \varepsilon.$$

We then claim, as before, that there exists some fixing of x_1 under which P also predicts $f(x_0x_2)$ with the same advantage. I.e., noting that:

$$\Pr_{x_0x_1x_2}[P(x_0, x_1, x_2, f(x_0x_1)) = f(x_0x_2)] = \mathsf{E}_{x_1}[\Pr_{x_0x_2}[P(x_0, x_1, x_2, f(x_0x_1)) = f(x_0x_2) \mid x_1]]$$

since the value is bigger than $\frac{1}{2} + \varepsilon$, and we're taking an average, there has to exist at least one fixed value x_1 such that the probability inside the expectation is bigger than $\frac{1}{2} + \varepsilon$. This is the fixing we choose. *This is called an "averaging argument"*.

Ok, then we define the circuit C to approximate f as follows: on input x_0x_2, the circuit C first computes $f(x_0x_1)$, for the particular fixing of x_1 that we chose above, and then outputs $P(x_0x_1x_2f(x_0x_1))$. So the circuit looks like:

(drawing in board)

The same inequality above now says that $\Pr[C(x_0x_2) = f(x_0x_2)] \geq \frac{1}{2} + \varepsilon$.

But we have $\mathsf{H}_{\mathsf{avg}}(f) \geq n^4$, so this is a contradiction unless $|C| \geq n^4$ or $\varepsilon < n^{-4}$.

The NW PRG

Ok, the idea is now: (picture in board) we take an input x with $\ell \gg n$ bits (think of ℓ as roughly quadratic in n), and we apply f several times, to different choices of n bits out of ℓ.

The object that chooses which bits to apply to is called a design.

Definition. Let d, n, ℓ satisfy $\ell > n > d$. A family $I = \{I_1, \ldots, I_m\}$ of subsets of $[\ell]$ is called an (ℓ, n, d)-design if $|I_j| = n$ for every j and $|I_j \cap I_k| \leq d$ for every $j \neq k$.

To derandomize BPP, we're gonna use designs with the following parameters:

Lemma. Construction of designs. There exists an algorithm A which on input (ℓ, d, n) with $n > d$ and $\ell > 10n^2/d$ runs for $2^{O(\ell)}$ steps and outputs an (ℓ, n, d)-design I containing $2^{d/10}$ subsets of $[\ell]$.

proof after

Now the construction:

The Nisan-Wigderson PRG. Take an (ℓ, n, d) design with $|I| = 2^{d/10}$ and suppose we have $f : \{0,1\}^n \to \{0,1\}$ with $\mathsf{H}_{\mathsf{avg}}(f) \geq 2^{2d}$. Then define the generator:

$$G : \{0,1\}^\ell \to \{0,1\}^{|I|}$$
$$G(x) = f(x_{I_1}) \cdots f(x_{I_m})$$

i.e., the i-th output bit of G is f applied to the input bits in the set I_i.

The theorem claims that this is a $(\mathsf{H}_{\mathsf{avg}}(f)/10, 1/10)$-secure PRG.

Proof. We need only prove that each output bit of G is $\frac{1}{2} + \frac{1}{10|I|}$-unpredictable by circuits of size $\mathsf{H}_{\mathsf{avg}}(f)/2$ or so.

As always, we work by contrapositive: we suppose that we could predict some bit of G and then we use the assumed predictor to compute f on average.

So suppose that some output bit of G has a $1/10$-distinguisher of size $\mathsf{H}_{\mathsf{avg}}(f)/10$. Then, by Yao's unpredictability theorem, there exists some circuit P of size $\mathsf{H}_{\mathsf{avg}}(f)/3$ with

$$\Pr_{x \in \{0,1\}^\ell}[P(G(x)_1, \ldots, G(x)_{i-1}) = G(x)_i] \geq \frac{1}{2} + \frac{1}{10|I|}.$$

But we know that $G(x)_j = f(x_{I_j})$. Let us rewrite the above inequality as follows. Think of the input $x \in \{0,1\}^\ell$ as being divided into two parts: the bits $x^0 = x_{I_i}$ which are given to the i-th copy of the function f, and the remaining bits $x^1 = x_{[\ell] \setminus I_i} = x_{[\ell] \setminus I_i}$.

Let us write $f_j(x^0 x^1) = f((x^0 x^1)_{I_j}) = G(x^0 x^1)_j$ to denote the j-th output bit of G. Note that $f_i(x^0 x^1) = f(x^0)$.

Then we can rewrite the above as

$$\Pr_{x^0 x^1}[P(f_1(x^0 x^1), \ldots, f_{i-1}(x^0 x^1)) = f(x^0)] \geq \frac{1}{2} + \frac{1}{10|I|}$$

Now, by an averaging argument as before, there has to exist a **fixed setting of** x^1 (i.e., a fixed setting of all the bits except those in I_i) that attains the same inequality. I.e., for this fixed choice of x_1,

$$\Pr_{x^0}[P(f_1(x^0x^1),\ldots,f_{i-1}(x^0x^1)) = f(x^0)] \geq \frac{1}{2} + \frac{1}{10|I|}$$

Ok, now let us build a circuit C for computing f on average. The circuit C is given as input x^0, computes $f_j(x^0x^1)$ *for the fixed choice* of x_1, for every $j = 1,\ldots,i-1$, and gives this as input to a circuit for computing P. The end result is:

$$C(x^0) = P(f_1(x^0x^1),\ldots,f_{i-1}(x^0x^1)).$$

The above inequality states, then, that this circuit C has some advantage in computing f. It then follows that either $|C| \geq \mathsf{H}_{\mathsf{avg}}(f)$ or $\frac{1}{10|I|} \leq \frac{1}{\mathsf{H}_{\mathsf{avg}}(f)}$.

But what is the size of C? Because every f_j with $j \neq i$ depends only on d bit positions in x^0, we can compute $f_i(x^0x^1)$ by a circuit of size $\leq 2^d$. Hence we can implement C using size

$$|C| \leq |P| + m2^d \leq \mathsf{H}_{\mathsf{avg}}(f)/3 + \mathsf{H}_{\mathsf{avg}}(f)/3 < \mathsf{H}_{\mathsf{avg}}(f)$$

(we had assumed that $\mathsf{H}_{\mathsf{avg}}(f) = 2^{2d}$). This is a contradiction, because in our design we had $|I| \leq 2^{d/10}$, which is also much smaller than $\mathsf{H}_{\mathsf{avg}}(f)$.

Putting it all together.

Theorem (NW '89?). If there exists a function $f \in \mathsf{E} = \mathsf{DTIME}(2^{O(n)})$ such that $\mathsf{H}_{\mathsf{avg}}(f) \geq 2^{\delta n}$ for some $\delta > 0$, then $\mathsf{P} = \mathsf{BPP}$.

Proof.

Since $\mathsf{H}_{\mathsf{avg}}(f) \geq 2^{\delta n}$, we take a design with $d = \frac{1}{2}\delta n$, $\ell = \frac{20}{\delta}n$. By the previous theorem, the NW generator G using this f we can output $2^{\frac{1}{20}\delta n}$ bits that cannot be distinguished from random by circuits of size $\leq 2^{\delta n}/10$.

Since we are in the derandomization regime, $G(x)$ can be computed in time $2^{O(n)}$:

- We first compute the design, which tells us the sets I_j where to apply f. This takes time $2^{O(\ell)} = 2^{O(n)}$.
- We now apply f to each copy, which takes per copy, and there are $2^{O(n)}$ copies of f being computed.
- Hence the total computation time is $2^{O(n)}$.

So how do we now fool a polynomial-time randomized algorithm?

Let us rescale the parameters: the above generator fools circuits of size $2^{\delta n}/10$ and outputs $2^{\frac{\delta}{20}n}$ bits and runs in time $2^{O(n)}$. If we let $N = 2^n$, we can rewrite this by saying that the above generator fools circuits of size N^δ, outputs $N^{\delta/10}$ bits, and runs in time $N^{O(1)}$. I.e., the generator is a polynomial-in-N-time algorithm.

If we want to fool an algorithm running in time n^c (this is now the input size of the BPP algorithm), we can invoke the above generator with parameter $N = n^{2c/\delta}$, and it will fool the circuit that simulates the BPP algorithm, hence it will also fool the algorithm itself.

That's the proof.

How to build designs.

There are a few ways. In the book it gives a greedy algorithm. I will have you study it for homework.

Getting the same result under worst-case hardness

I will briefly sketch the proof of the later result by Impagliazzo and Wigderson:

Theorem (IW'97). If $\mathsf{E} \not\subseteq SIZE(2^{\delta n})$ for some $\delta > 0$, then $\mathsf{P} = \mathsf{BPP}$.

Theorem (IW'97). If there exists some $f \in \mathsf{DTIME}(2^{O(n)})$ with $\mathsf{SIZE}(f) \geq 2^{\delta n}$ for some $\delta > 0$, then there exists some $f' \in \mathsf{DTIME}(2^{O(n)})$ such that $\mathsf{H}_{\mathsf{avg}}(f') \geq 2^{\delta' n}$ for some $\delta' > 0$ (smaller than δ).

The proof uses locally-list-decodable codes, which is a primitive from information theory, more precisely from the theory of error-correcting codes.

Definition. An (n, m, ε)-locally-list-decodable code is a $\mathsf{poly}(n)$-time computable map $E : \{0,1\}^n \to \{0,1\}^m$, together with a $\mathsf{polylog}(m)$-time oracle Turing machine D such that:

$$\forall x \in \{0,1\}^n \forall E' \in \{0,1\}^m \qquad \Pr_i[E'_i = E(x)_i] \geq \frac{1}{2} + \varepsilon \implies$$
$$\exists i \in [\mathsf{poly}(n, 1/\varepsilon)] \forall j \in n \qquad \Pr[D^{E'}(0^n, i, j) = x_j] \geq 2/3$$

I.e.

1. We have a way of encoding $x \in \{0,1\}^n$ into a codeword $E(x) \in \{0,1\}^m$, such that
2. We can change almost $1/2$ of all the positions in $E(x)$ to obtain a corrupted codeword E' and
3. it will always be possible to recover from E' a list of $\mathsf{poly}(n)$ candidates $x^1, \ldots, x^{n^c}$, such that $x = x^j$ for some j, and furthermore
4. There exists an algorithm D which on input i, j with oracle access to E' can output x^i_j. And it runs in time $\mathsf{poly}(|i|, |j|)$, i.e. in time $\mathsf{polylog}(|E|)$

Now we can prove the above theorem. The idea is as follows.

Suppose we have a function $f : \{0,1\}^N \to \{0,1\}$ which does not have circuits of size $2^{\delta n}$.

Think of f as a string of length $n = 2^N$. The string is $x_1 \ldots x_n = f(0^N) \ldots f(1^N)$

Now define $f'(j)$, for $j \in \{0,1\}^{\log m}$ by $f'(j) = E(x)_j$.

We claim that $\mathsf{H}_{\mathsf{avg}}(f') \geq 2^{\delta' n}$ for some $\delta' > 0$.

Suppose otherwise. I.e. we have some circuit C of size $\leq 2^{\delta' n}$ which computes f on average.

That means that the string $E' = C(0^{\log m}) \ldots C(1^{\log m})$ has $\Pr[E'_i = f'(i)] \geq \frac{1}{2} + 2^{-\delta' n}$.

I.e., the truth table of our circuit C is a corrupted version of the truth table of f', which itself is a codeword of our locally-decodable code.

But then we have that, for some choice i, the decoder $D^{E'}(0^n, i, j) = f(j)$ with probability at least 2/3. By repeating the decoder using indepenent randomness, we can compute $f(j)$ with probability $\geq 2^{-2N}$ or so. But then by union bound there exists some fixing of the randomness that computes $f(j)$ correctly on all j.

You can read about LLDCs in chapter 19 (?).